GQR — Gerador de QR Code

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	Gerador de QR Code
Gerente de Projeto	Julia de Souza Seger
Gerente/Responsável de GCS-DevOps	Julia de Souza Seger (Desenvolvedora Líder; o par de revisão é Iasmin (colega))
Fábrica de Software / Squad	Individual, com par de revisão (Iasmin) — sem squad
Repositório principal	https://github.com/juliasgr/Aula-mobile-Gerador-QRCode

Histórico de Revisões

Data	Versão	Descrição	Autor
24/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Julia Seger
21/10/2026	1.1	Melhoria do documento pós alteração nas branches e configurações do repositório.	Julia Seger

Sumário

Sumário..... 2

1. Introdução..... 3
1.1 Finalidade..... 3
1.2 Escopo..... 3
1.3 Definições, Acrônimos e Abreviações..... 3
1.4 Referências..... 3
1.5 Visão Geral..... 3
2. Cultura DevOps e Papéis na Gerência de Configuração..... 4
3. Ferramentas, Ambientes e Infraestrutura..... 4
3.1 Ferramentas do Pipeline..... 4
3.2 Ambientes e Infraestrutura..... 5
3.3 Estratégia de Branching e Versionamento..... 5
4. Identificação da Configuração..... 6
4.1 Itens de Configuração..... 6
4.2 Convenção de Nomenclatura..... 6
4.3 Estrutura do Repositório..... 6
5. Integração Contínua (CI)..... 6
5.1 Etapas do Pipeline de CI..... 6
5.2 Política de Pull Request e Revisão de Código..... 7
5.3 Critérios de Qualidade (Quality Gate)..... 7
6. Entrega e Implantação Contínua (CD)..... 7
6.1 Etapas do Pipeline de CD..... 7
6.2 Estratégias de Implantação..... 7
6.3 Rollback e Recuperação..... 7
7. Infraestrutura como Código (IaC)..... 7
7.1 Versionamento de Infraestrutura..... 8
7.2 Ambientes Efêmeros..... 8
8. Controle de Configuração e Mudança..... 8
8.1 Processo de Solicitação de Mudança..... 8
8.2 Comitê de Controle de Mudança (CCM)..... 8
9. Baselines e Releases..... 8
9.1 Geração Automática de Baseline..... 8
9.2 Notas de Versão (Release Notes)..... 8
10. Estimativa do Status de Configuração..... 9
10.1 Monitoramento e Observabilidade..... 9
10.2 Métricas DORA..... 9
10.3 Relatórios e Auditorias..... 9
11. Marcos..... 9
12. Treinamento e Recursos..... 9
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros.. 9
14. Quadro-Resumo do Pipeline CI/CD..... 9

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado à autora individual do projeto, Julia de Souza Seger, responsável pelo desenvolvimento do sistema "Gerador de QR Code" que é um aplicativo desenvolvido em Flutter (Dart) como projeto da disciplina de desenvolvimento mobile, executado localmente em ambiente de testes via flutter run (Chrome/web), sem implantação em ambiente de produção até o momento.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

Repositório do projeto: https://github.com/juliasgr/Aula-mobile-Gerador-QRCode

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado,

testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Líder do projeto: integra código com frequência na branch principal, escreve testes automatizados, abre Pull Requests e é corresponsável pela qualidade do que é implantado. Aprova as implantações em produção (Environment production).
Par de revisão (Iasmin)	Colega com acesso de escrita ao repositório (usuário GitHub: https://github.com/Iasminkrueger). Revisa e aprova os Pull Requests da Desenvolvedora Líder (o GitHub não permite aprovar o próprio PR); em contrapartida, Julia revisa o projeto desse colega.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, IaC e pipelines como (build + testes verdes) código
Orquestração de CI/CD	GitHub Actions (a implementar)	Automação de build, testes, análise e implantação
Empacotamento	Flutter build (APK/AAB para Android, build web para navegador)	Geração de imagens/artefatos versionados e imutáveis

Registro de artefatos	GitHub Releases (a implementar)	Armazenamento e versionamento de imagens e pacotes
Infraestrutura como Código	N/A — projeto não possui backend/infraestrutura própria	Provisionamento e configuração versionada de ambientes
Orquestração de containers	N/A — não se aplica a aplicativo Flutter mobile/web sem backend	Execução e escalonamento dos serviços implantados
Gestão de segredos	GitHub Secrets (a implementar, se necessário)	Armazenamento seguro de credenciais usadas nos pipelines
Monitoramento e Observabilidade	N/A — sem ambiente de produção implantado	Métricas, logs e alertas dos ambientes
Gestão de dependências / SCA – *Software Component Analisys*	Dependabot (alertas de segurança do GitHub)	Identificação de vulnerabilidades em bibliotecas de terceiros
Rastreamento de tarefas	GitHub Issues	Vinculação de commits/PRs a itens de trabalho

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Execução local via flutter run (Chrome/emulador) durante o desenvolvimento.	Alterações locais no código
Homologação / Staging	Ambiente de homologação configurado no GitHub (Settings → Environments → staging), restrito a branches protegidas. Recebe automaticamente a versão gerada após o merge na main (a automação entra na Aula 3).	Merge na main
Produção	Ambiente configurado no GitHub (Environment production), com a Desenvolvedora Líder como revisora obrigatória (sem a opção Prevent self-review) e restrito a branches protegidas. Publicação somente após aprovação manual.	Aprovação da Desenvolvedora Líder no Environment production

Nota: os Environments staging e production já estão configurados no repositório (Aula 1). O pipeline de CI (Aula 2) e o de CD, que fará os deploys nesses ambientes (Aula 3), ainda serão implementados; até lá, as práticas de CI/CD descritas neste documento representam o modelo-alvo.

3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Nota: a branch main é protegida no GitHub: não aceita commits diretos, exige Pull Request com 1 aprovação do par de revisão, descarta aprovações antigas quando há novos commits, exige histórico linear e não permite exceções (nem para a administradora). Os merges são feitos somente por squash merge, com o título e a descrição do PR como mensagem do commit, e as branches são excluídas automaticamente após o merge. A exigência de status checks (CI verde) será adicionada na Aula 2.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Dockerfiles e manifests de containers — não aplicável a este projeto (aplicativo Flutter sem containerização)
- README.md (o que o app faz, stack, como instalar, executar e testar), .gitignore e .gitattributes (* text=auto eol=lf)
- Lockfile de dependências (pubspec.lock) e .env.example (somente nomes das variáveis de ambiente, sem valores reais)
- Modelos de colaboração em .github/: pull_request_template.md, CODEOWNERS e ISSUE_TEMPLATE (funcionalidade.yml e defeito.yml)
- Segredos: se alguma senha, token ou chave for commitada, ela permanece no histórico mesmo após apagar o arquivo; deve-se trocar a credencial no serviço de origem e passar a usar variáveis de ambiente e GitHub Secrets
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes) — não aplicável a este projeto (sem infraestrutura própria)
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.
- Scripts de migração de banco de dados (N/A)
- Documentação técnica (README, ADRs — Architecture Decision Records)

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<ticket>-<descrição-curta>	feature/SIQQ-123-login-oauth
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(auth): adiciona login via OAuth2
Tag de release	Versionamento Semântico vX.Y.Z	v2.3.0
Artefato de release	<projeto>-vX.Y.Z.<extensão>	gerador-qrcode-v1.0.0.apk
...	...	...

4.3 Estrutura do Repositório

```
qr_gerador_flutter/
├── pubspec.yaml              # dependências (bibliotecas do pub.dev)
├── pubspec.lock              # versões exatas das dependências
├── README.md
├── .gitignore
├── .gitattributes
├── .env.example
├── .github/
│   ├── CODEOWNERS
│   ├── pull_request_template.md
│   └── ISSUE_TEMPLATE/ (funcionalidade.yml, defeito.yml)
├── lib/
│   ├── main.dart             # ponto de entrada e tema do app
│   ├── models/
│   │   └── qr_history_item.dart      # modelo de um item do histórico
│   ├── services/
│   │   └── history_service.dart      # salvar/ler histórico (shared_preferences)
│   ├── screens/
│   │   └── home_screen.dart          # tela principal (campo + QR + histórico)
│   └── widgets/
│       └── history_tile.dart # item visual da lista de histórico
```

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (com cache).
2. Build da aplicação.
3. Análise estática de código (lint) e verificação de padrões de codificação.
4. Testes unitários, com medição de cobertura mínima.
5. Testes de integração.
6. Análise de segurança do código (SAST) e de dependências (SCA).
7. Empacotamento e publicação do artefato/imagem versionado no registro.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação do par de revisão (colega com acesso de escrita) antes do merge ("quatro olhos"); novos commits no PR descartam aprovações anteriores. Commits diretos na main são bloqueados, inclusive para a administradora.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente por squash merge e, a partir da Aula 2, somente com pipeline de CI verde (quality gate).

5.3 Critérios de Qualidade (Quality Gate)

Build/lint sem erros, testes manuais das funcionalidades principais (gerar QR, salvar/ler histórico) antes do merge.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

Nota: até o momento, o projeto não possui pipeline de CD implementado nem ambiente de produção publicado. As etapas abaixo descrevem o modelo-alvo a ser adotado caso o projeto evolua para publicação (ex.: geração de APK assinado, distribuição via loja ou serviço de distribuição interna).

6.1 Etapas do Pipeline de CD

8. Build do artefato (APK/AAB para Android ou build web) a partir da versão validada em CI.
9. Execução de testes automatizados de aceitação/widget tests, quando aplicável.
10. Gate de aprovação para publicação (manual, dado o caráter individual do projeto).
11. Geração de artefato assinado e publicação (ex.: GitHub Releases, loja de aplicativos ou serviço de distribuição interna — a definir).
12. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Distribuição por versão (release-based)	Cada versão estável é publicada como um artefato único (ex.: APK), sem coexistência de múltiplas versões em produção simultaneamente.
Feature Flags	Funcionalidades podem ser implantadas desligadas e ativadas posteriormente de forma independente do release, reduzindo o risco de novas versões.

Nota: estratégias como Rolling Update, Blue-Green e Canary Release são aplicáveis a serviços web/backend com infraestrutura própria, o que não é o caso deste projeto (aplicativo Flutter sem backend ou ambiente de produção).

6.3 Rollback e Recuperação

Como o projeto não possui ambiente de produção ativo, o rollback se resume, no modelo-alvo, à possibilidade de reverter para uma versão anterior do artefato publicado (ex.: reinstalação de um APK de release anterior), com base no histórico de tags versionadas no repositório.

7. Infraestrutura como Código (IaC)

Não se aplica ao estado atual do projeto: por se tratar de um aplicativo Flutter sem backend ou infraestrutura própria (servidores, banco de dados, etc.), não há infraestrutura a ser provisionada ou versionada como código.

7.1 Versionamento de Infraestrutura

Não se aplica — ver justificativa na seção 7.

7.2 Ambientes Efêmeros

Não se aplica — ver justificativa na seção 7.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

Por ser um projeto individual, não há comitê formal: as decisões de mudança são da autora (Gerente de Projeto e Gerente de GCS-DevOps), mas nenhuma mudança entra na main sem a aprovação do par de revisão no Pull Request; o par de revisão atua como controle independente.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Neste plano, cada release publicado (ex.: artefato gerado a partir da branch principal) gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit correspondente e ao artefato publicado (quando aplicável). Como o projeto não possui ambiente de produção nem infraestrutura própria, a baseline não inclui estado de infraestrutura.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Não se aplica no momento, uma vez que o projeto não possui ambiente de produção implantado. Caso o aplicativo venha a ser publicado, recomenda-se a adoção de ferramentas de monitoramento de crashes e uso (ex.: Firebase Crashlytics, Firebase Analytics), comuns em aplicações Flutter.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

Marcos sugeridos para este projeto: implantação do pipeline de CI (build, lint, testes automatizados); primeira geração automática de artefato versionado (APK/build); primeira publicação de release no GitHub Releases; eventual publicação em loja de aplicativos (se aplicável no futuro).

12. Treinamento e Recursos

Por se tratar de um projeto individual, o treinamento se resume ao autoaprendizado da autora em Git/GitHub (versionamento e Pull Requests) e no framework Flutter/Dart (desenvolvimento e testes locais via flutter run), com apoio da documentação oficial de ambas as ferramentas.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

Não se aplica — projeto individual, sem subcontratados ou fornecedores externos de desenvolvimento.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de segurança, publicação do artefato.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Build de Release	Build do artefato final (APK/AAB ou build web) a partir da versão validada em CI.	Merge na branch principal
Gate de Release	Aprovação para publicação (manual, dado o caráter individual do projeto).	Sucesso dos testes em staging
CD — Produção	Geração de artefato assinado, publicação (GitHub Releases ou loja/distribuição interna) e geração de baseline/tag.	Aprovação do gate de release